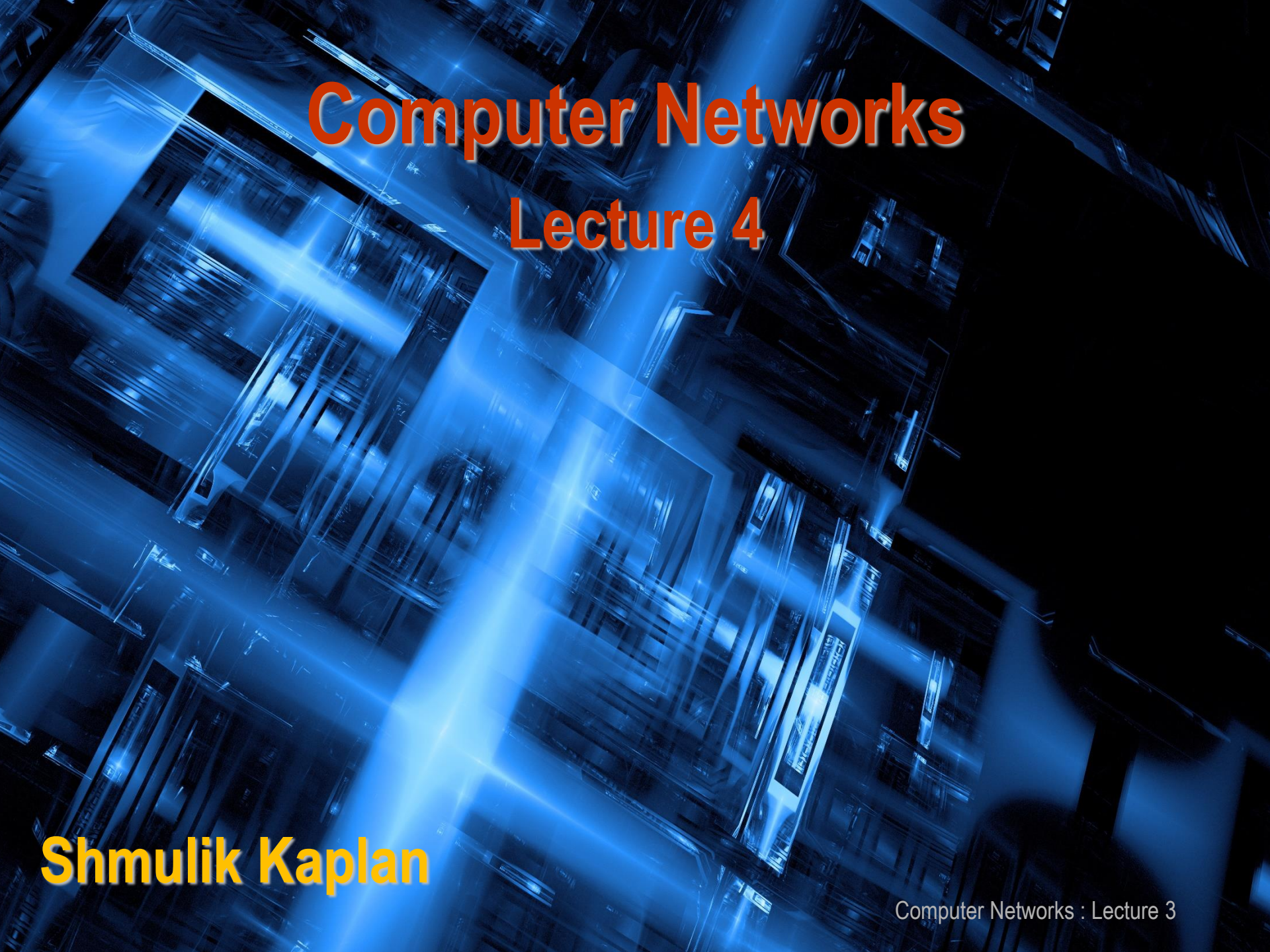


Computer Networks

Lecture 4

Shmulik Kaplan

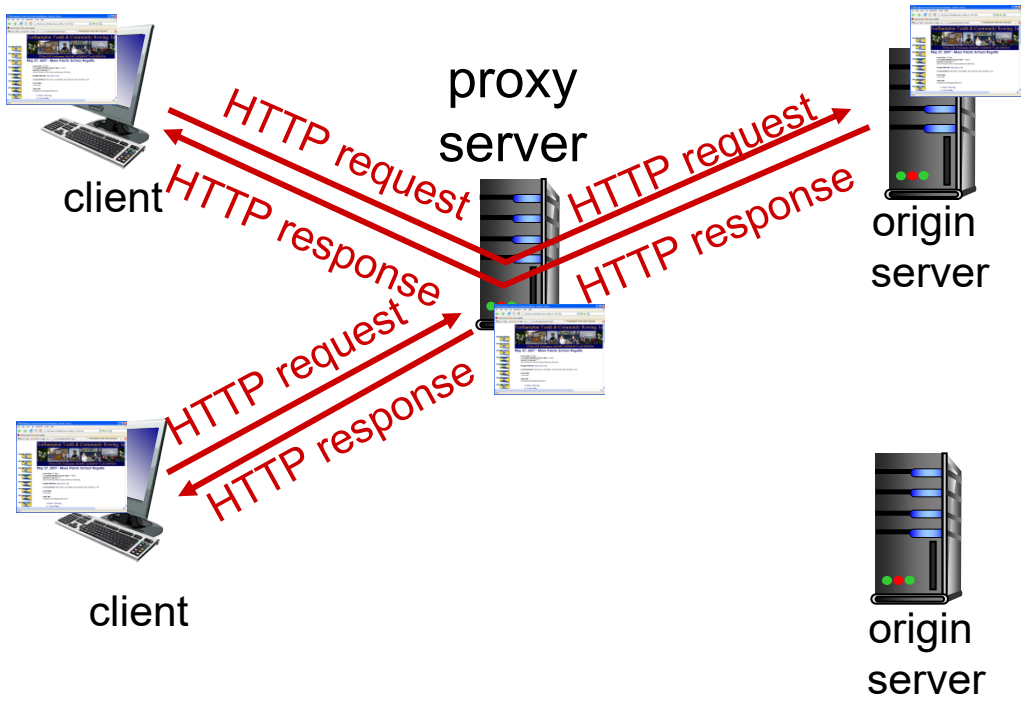
Today

- Web browsing protocol: HTTP
  - Proxy
  - Security
  - HTTP 2 & HTTP 3
- Mail Protocols:
  - SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

Web caches (proxy server)

Goal: satisfy client request without involving origin server

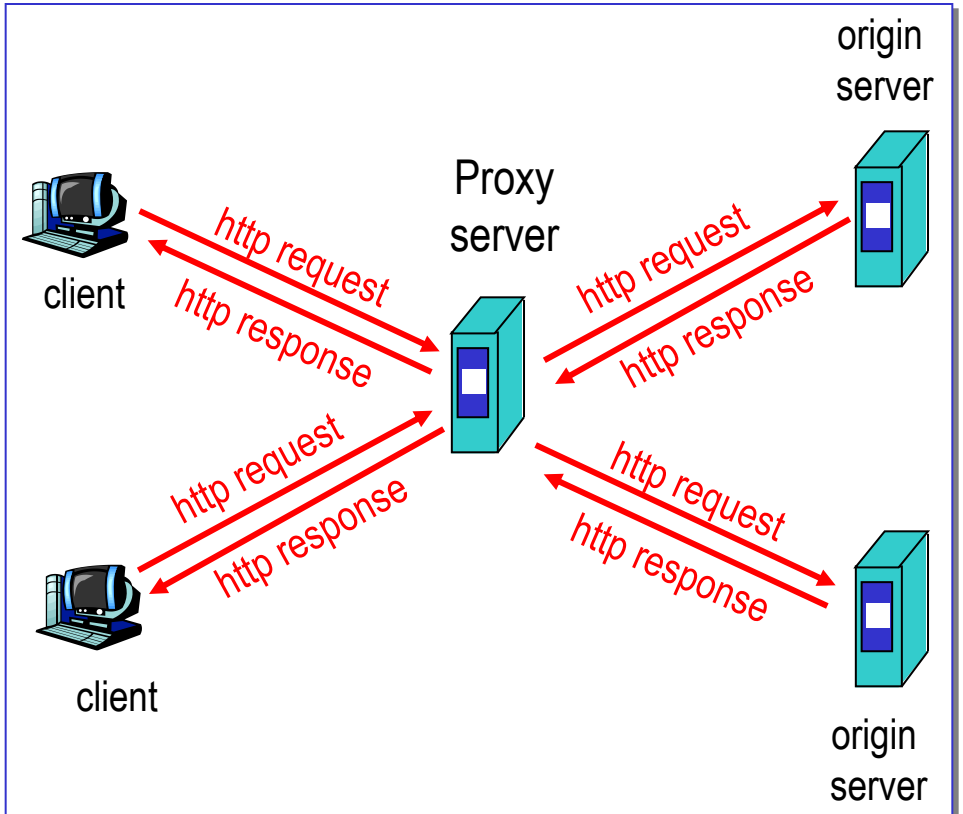
- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
  - object in cache: cache returns object
  - else cache requests object from origin server, then returns object to client



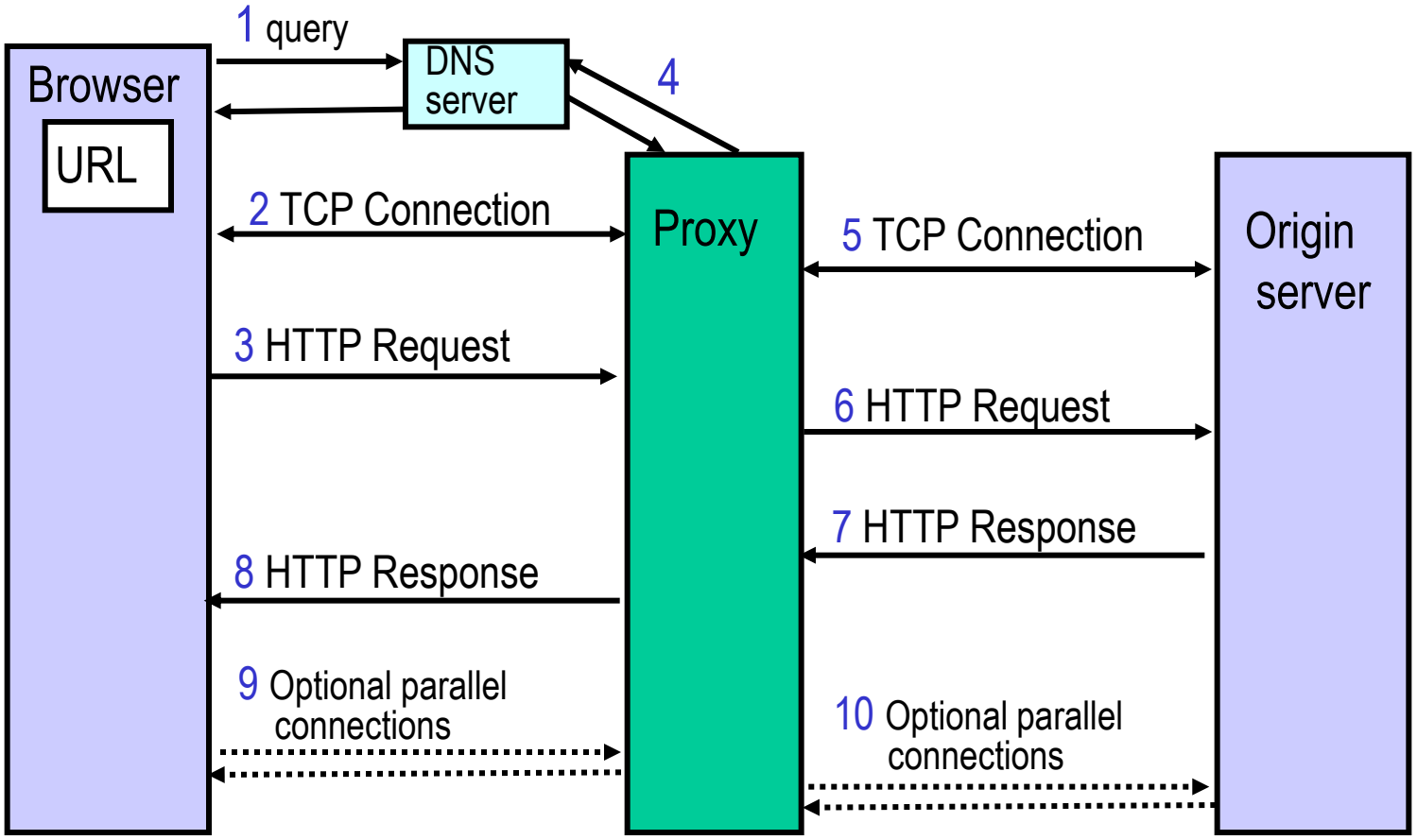
Proxy server

Proxy: A program in the path between the user agent and an origin server that functions as server to the user agent and as client to the origin server

- Caching responses
- Anonymizing clients
- Transforming requests and resp. (low-bandwidth,compression…)
- Gateway to non-HTTP systems
- Filtering requests and responses (URL filtering, searching, remove headers)
- Firewall



Steps in a browser process with proxy in the path




```
Can use also: CONNECT www.example.com:443 HTTP/1.1
(Only the initial connection request is HTTP)
```

Today

- Web browsing protocol: HTTP
  - Proxy
  - Security
  - HTTP 2 & HTTP 3
- Mail Protocols:
  - SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

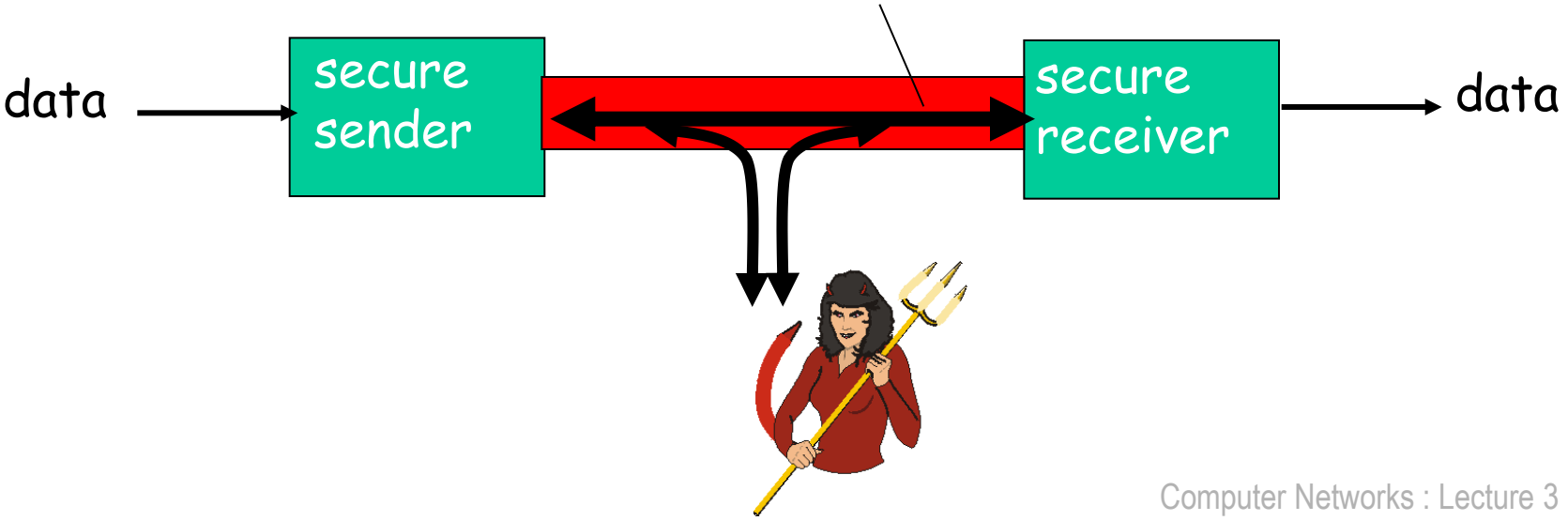
What is network security?

Confidentiality: (סודיות, חשאיות) only sender, intended receiver should "understand" message contents

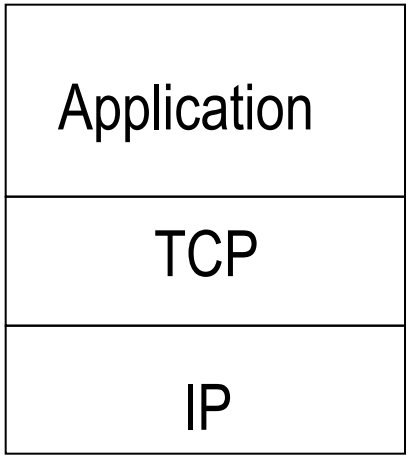
- sender encrypts message
- receiver decrypts message

Authentication: (אימות) sender, receiver want to confirm identity of each other

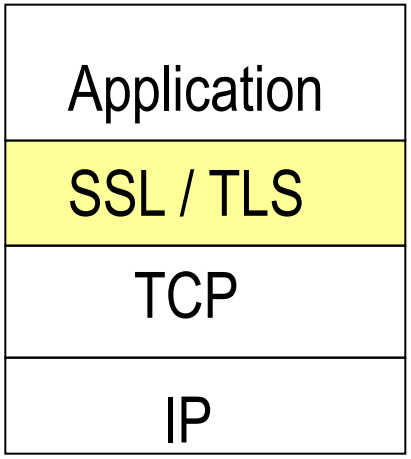
Message integrity: (שלמות) sender, receiver want to ensure message not altered (in transit, or afterwards) without detection.



SSL: Secure Sockets Layer
TLS: Transport Layer Security



Normal Application



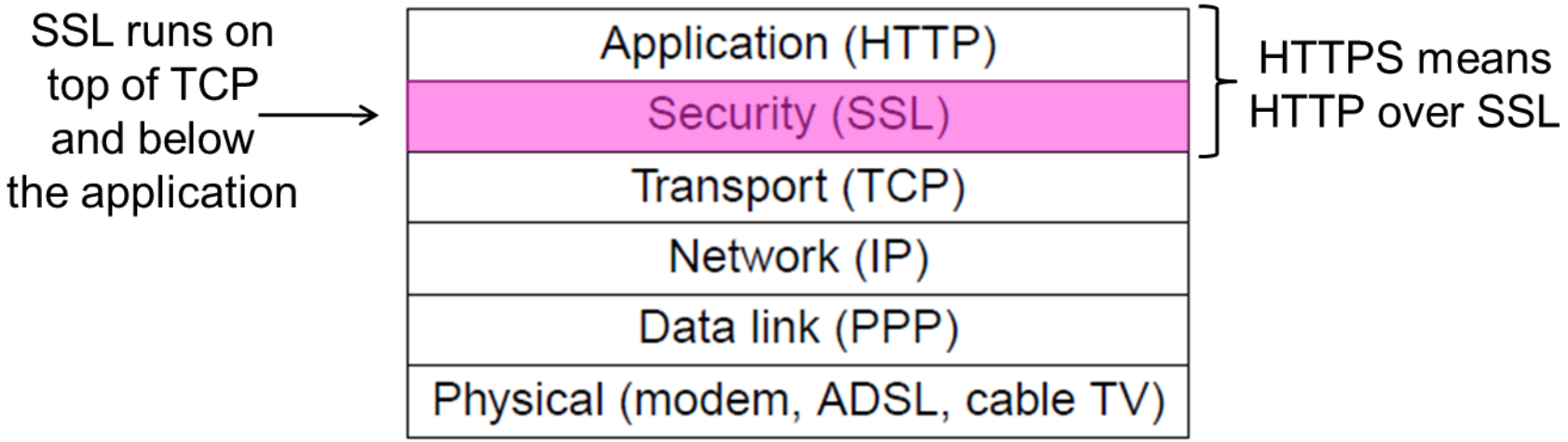
Application with SSL

- SSL/TLS provides application programming interface (API) to applications
- C and Java SSL/TLS libraries/classes readily available
- HTTP which uses SSL/TLS is called HTTPs

SSL/TLS

- Widely deployed security protocol
  - supported by almost all browsers, web servers
- Available to all TCP applications

- Provides
  - *confidentiality*
  - *authentication*
  - *integrity*



SSL in the protocol stack

Today

- Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - HTTP 2 & HTTP 3
- Mail Protocols:
  - SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

HTTP 1.1 vs 2 vs 3

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport Protocol	TCP	TCP	QUIC (over UDP)
Connection Setup	1 RTT for TCP + 1 RTT for TLS	1 RTT for TCP + 1 RTT for TLS	1 RTT (or 0-RTT) QUIC handshake
Multiplexing	Not really	Yes	Yes
Header Compression	None	HPACK	QPACK
Data Framing	Text-based	Binary frames	Binary frames
Encryption	Optional	Usually encrypted	Always TLS 1.3 built in
Prioritization	Limited	Stream prioritization	Stream prioritization
Loss Recovery	Handled by TCP	Handled by TCP	Handled per-stream by QUIC (no global stalls)
Performance	Baseline, sequential requests	Faster, fewer connections	Fastest, lower latency and mobile-friendly
Browser Support	Universal	Universal	Modern browsers (Chrome, Edge, Firefox, Safari)

Today

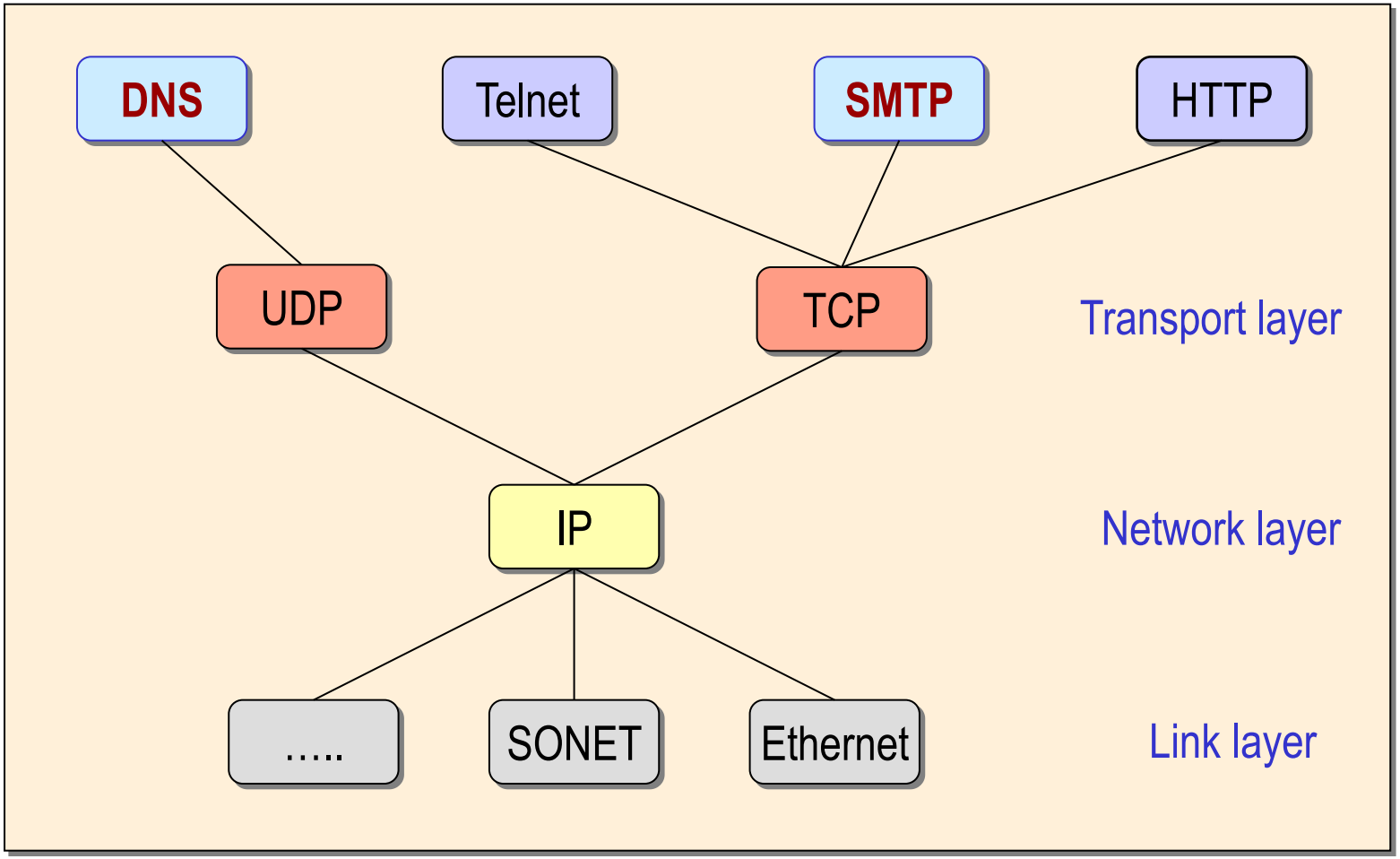
- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- Mail Protocols:
  - SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- Mail Protocols:
  - SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

Today:
- SMTP (POP, IMAP)
- DNS (DoT, DoH, CDN)

Computer Networks



Client Server - definition

Client Sever definition

- A **client** is a program that **initiates requests** for services or resources.
- A **server** is a program that **waits for incoming requests** and **provides services**

Properties

- Role asymmetry – client initiates – server waits
- Communication pattern – "Request–response"
- Separation of responsibilities – client = user interaction, server = storage, business logic
- Servers are shared
- Servers are persistent

Not required

- The server does *not* need to send data spontaneously
- The server does *not* need to know the identity of the client
- A client is not defined by being "weaker"
- Not necessarily HTTP

Electronic Mail

Three major components

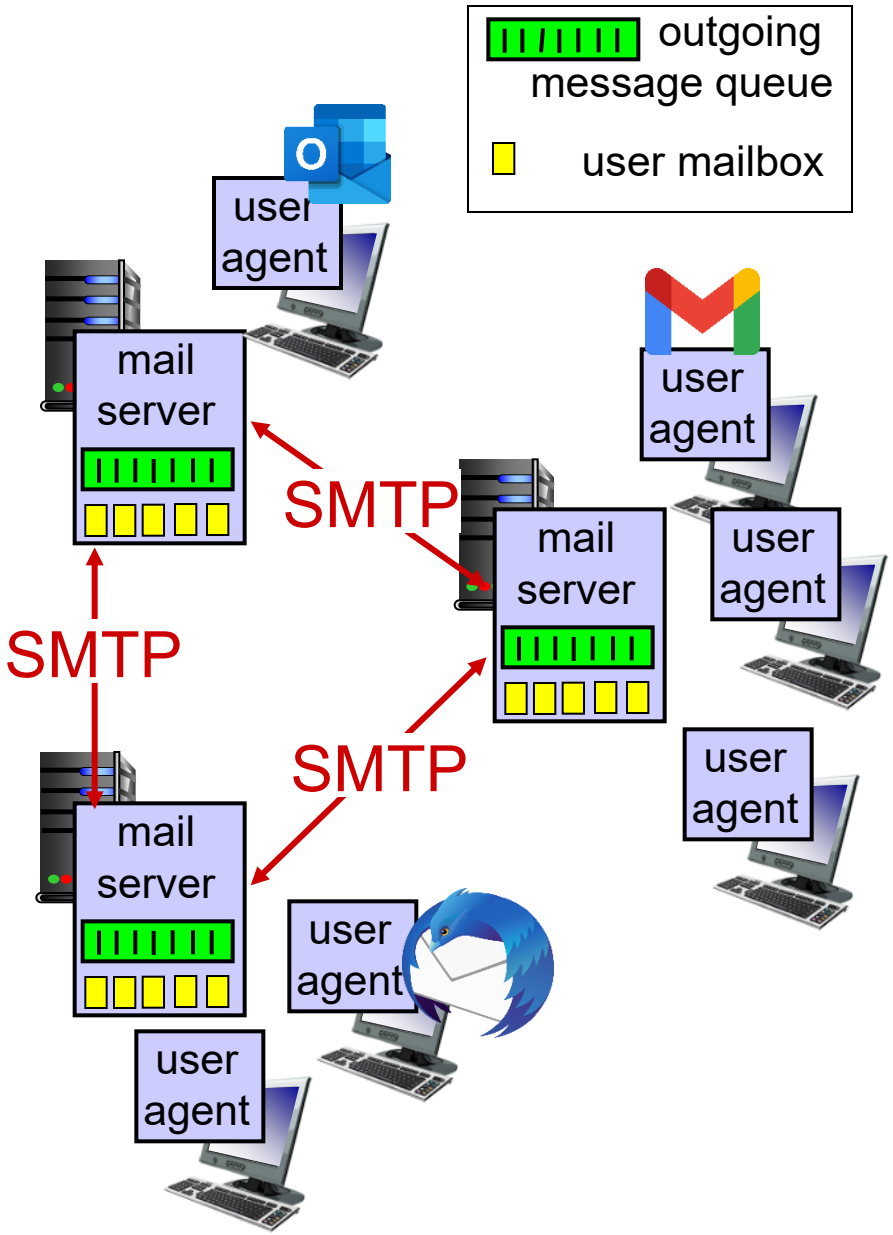
- User agents (i.e., outlook, gmail)
- Mail servers
- Simple Mail Transfer Protocol: SMTP

User Agent (mail reader)

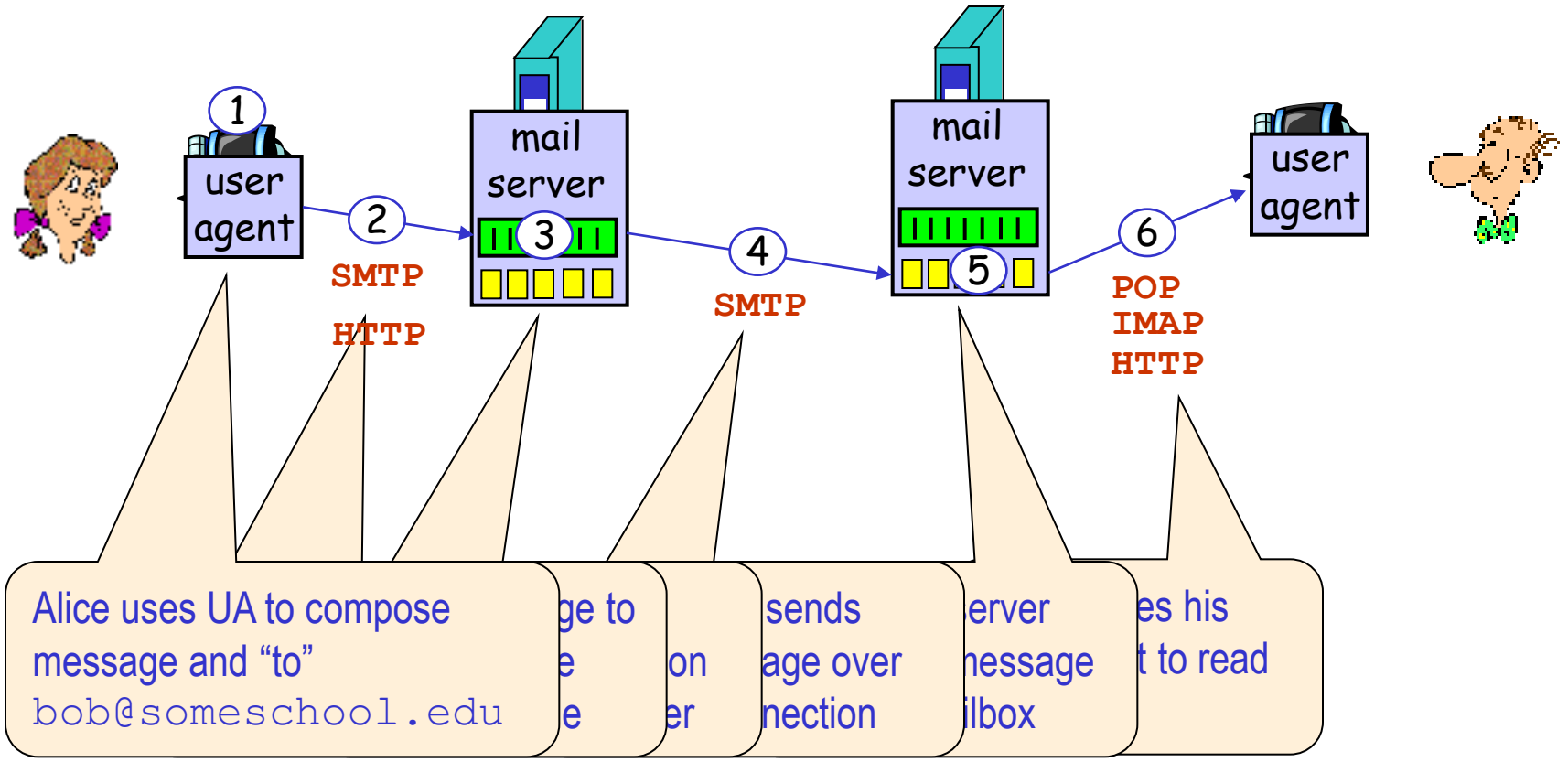
- composing, editing, reading mail messages

Mail Servers

- mailbox contains incoming messages (yet to be read) for user
- message queue of outgoing (to be sent) mail messages
- smtp protocol between mail servers to send email messages
  - client: sending mail to server
  - "server": receiving mail server



Scenario: Alice sends message to Bob



Simple Mail Transfer Protocol [RFC 821]

- Uses TCP to transfer email msg on port **25** (uses persistent connections)
- Three phases of transfer
  - handshaking (greeting)
  - transfer of messages
  - closure

- Command/response interaction
  - commands: ASCII text
  - response: status code and phrase
- smtp server uses `CRLF.CRLF` to determine end of message
- Messages must be in ASCII (header & body)
- Thus, message has to be encoded (base-64 or quoted printable)



SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "."
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

handshake

message

closure

Extended/Enhanced SMTP

Defined 1995, RFC 1869

ESMTP adds enhancements to the SMTP protocol which include security (250-STARTTLS), authentication, etc. …

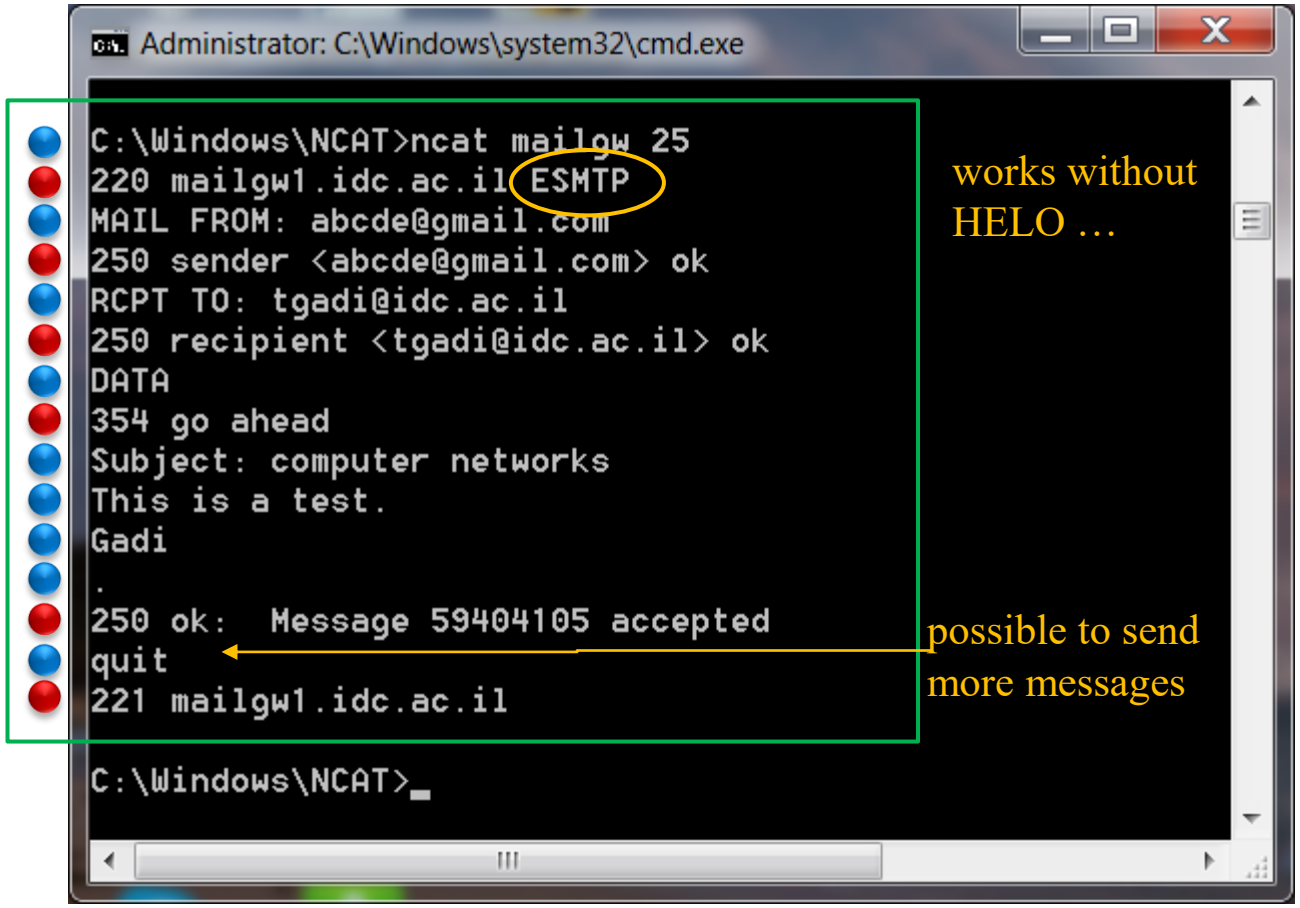
```
S: 220 hamburger.edu ESMTP
C: EHLO crepes.fr
S: 250-hamburger.edu
S: 250 …
S: 250-SIZE 10000000
S: 250-STARTTLS
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "."
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

handshake

message

closure

SMTP



You can use **telnet** instead of **ncat** in this example.

Try SMTP interaction for yourself

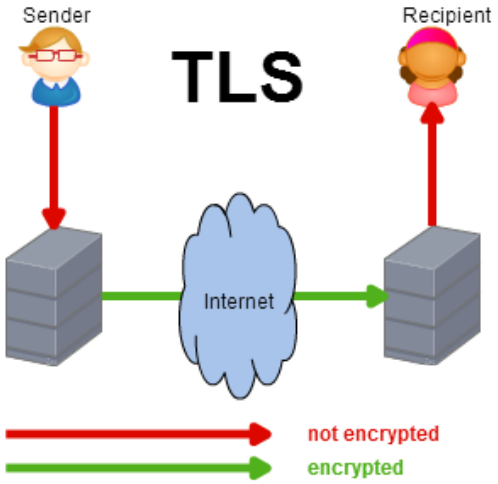
- **`telnet (or ncat) servername 25`**
- see 220 reply from server
- enter HELO, MAIL FROM, RCPT TO, DATA, QUIT

Commands above lets you send email without using email client (reader)

But only if you find an "Open Relay SMTP Server" …

- Abuse by spammers ➔ open relay blacklists
- Read more about open relay here.

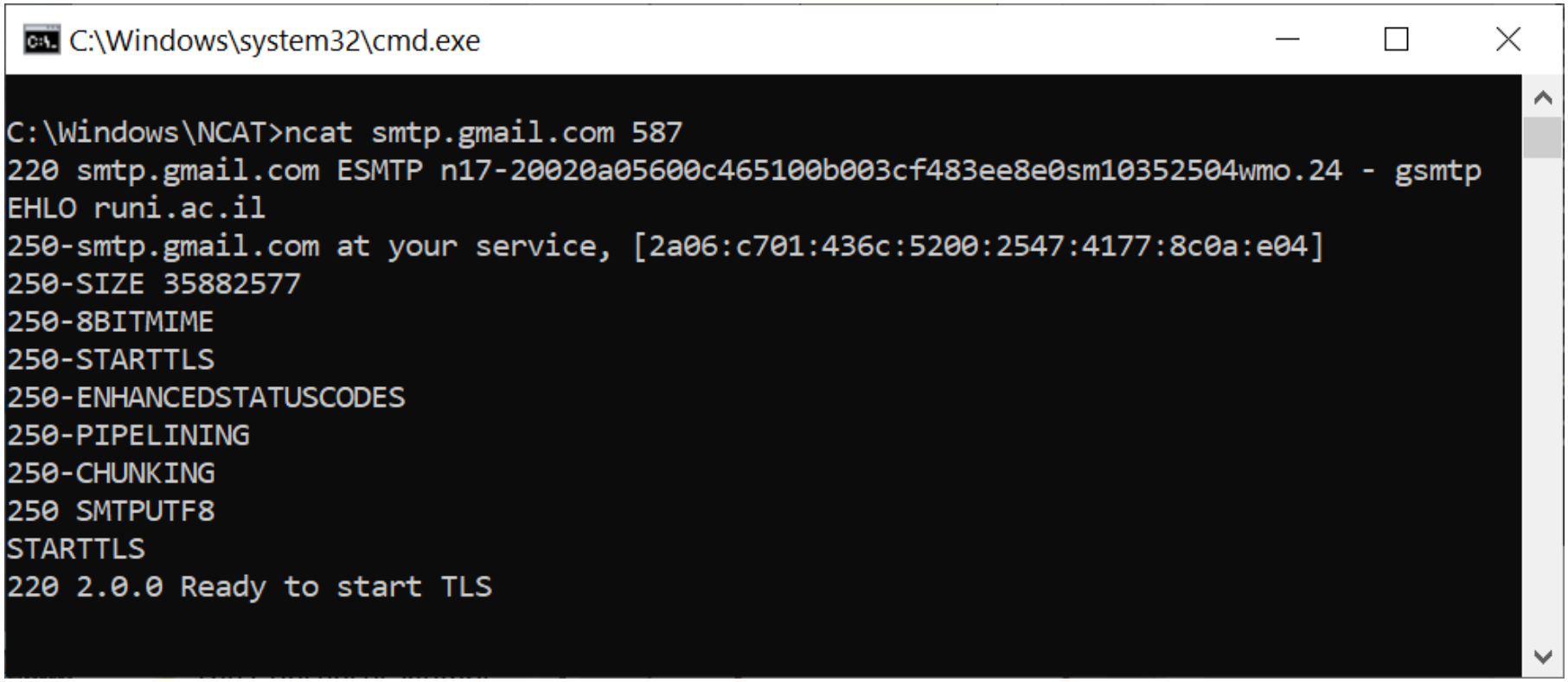
SMTP over SSL/TLS (SMTPs)



SMTP uses port 25, but SSL and TLS encrypted SMTP use ports 465 and 587, respectively.

Try: ncat smtp.gmail.com 587

SMTP over SSL/TLS (SMTPs)



Comparison with HTTP

- http: pull
- smtp: push

- http/1.1: have ASCII command/response interaction, status codes
- smtp: same as above

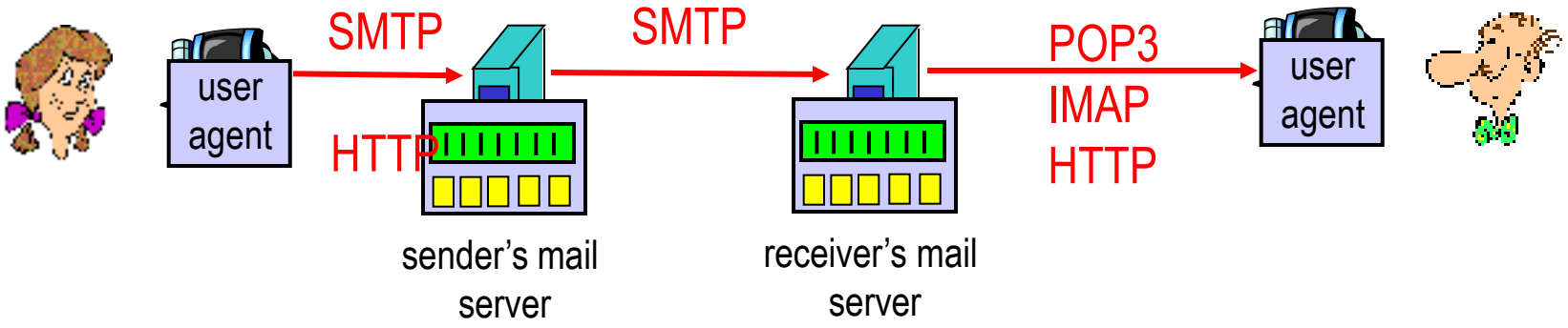
- http: response includes a **`Content-length:`** header
- smtp: the body ends with "**`CRLF.CRLF`**"

- http: each object is encapsulated in its own response message
- smtp: multiple objects message sent in a multipart message

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- Mail Protocols:
  - ✓ SMTP
  - POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

Mail access protocols



- SMTP: delivery/storage to receiver's server
- Mail access protocol: retrieval from server
  - POP: Post Office Protocol [RFC 1939]
    - authorization (agent <-->server) and download
  - IMAP: Internet Mail Access Protocol [RFC 1730, 2060]
    - more features (more complex)
    - manipulation of stored msgs on server
  - HTTP: Gmail, Outlook , Yahoo! Mail, etc.

POP3 protocol [RFC 1939]

port 110

authorization phase
- client commands
  - **user** declare username
  - **pass** password
- server responses
  - **+OK**
  - **-ERR**

transaction phase, client:
- **list:** list message numbers
- **retr:** retrieve message by number
- **dele:** delete
- **quit**

```
S: +OK POP3 server ready
C: user alice
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 2 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Try POP3 interaction for yourself

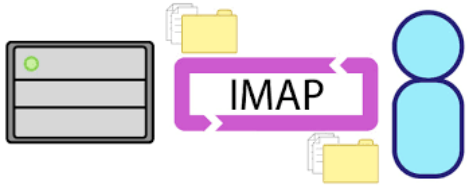


- `telnet servername 110`
- see `+OK POP3 server ready` from server
- enter USER, PASS, …… commands lets you read email without using email client (reader)

POP uses port 110, but SSL/TLS encrypted POP uses port 995

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN



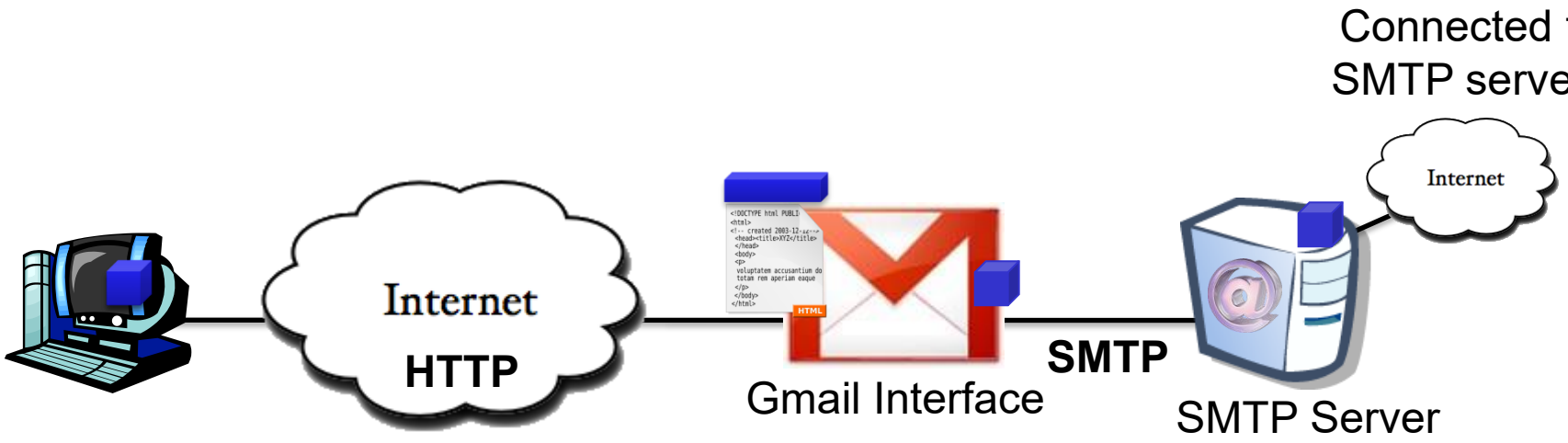
IMAP protocol [RFC 2060]

- Many more features than POP3, but complex
- Allow users to manipulate remote mailboxes (search)
- Maintain a folder hierarchy for each user
- Allow to obtain components of messages
- The IMAP server listens to port 143.

IMAP uses port 143, but SSL/TLS encrypted IMAP uses port 993

Web-based email

Outlook.com YAHOO! MAIL Gmail by Google



Dec 1995 – started
July 1996 – first version
18 months later 12M users …

msn Hotmail®

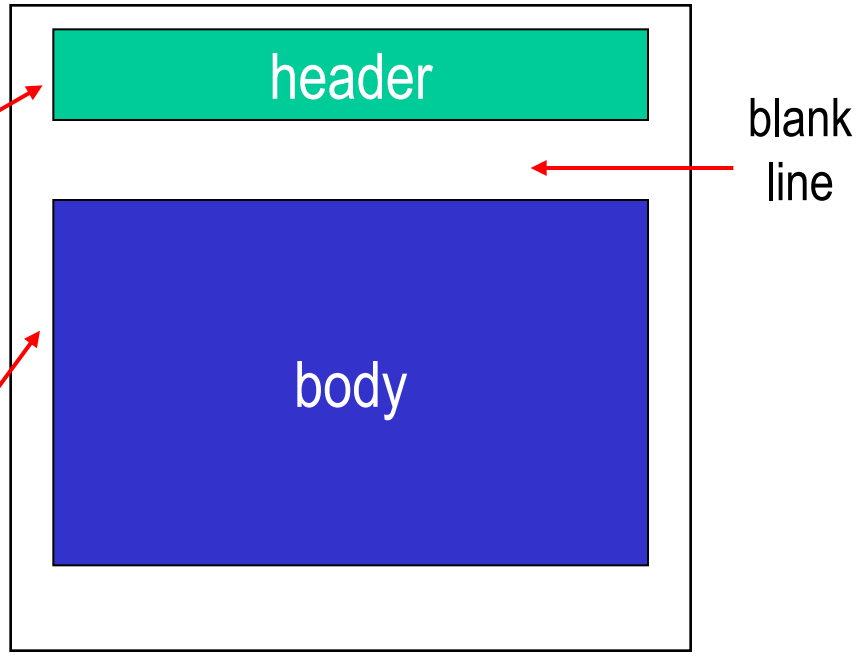
Mail message format

SMTP: protocol for exchanging email msgs

RFC 822: standard for text message format:

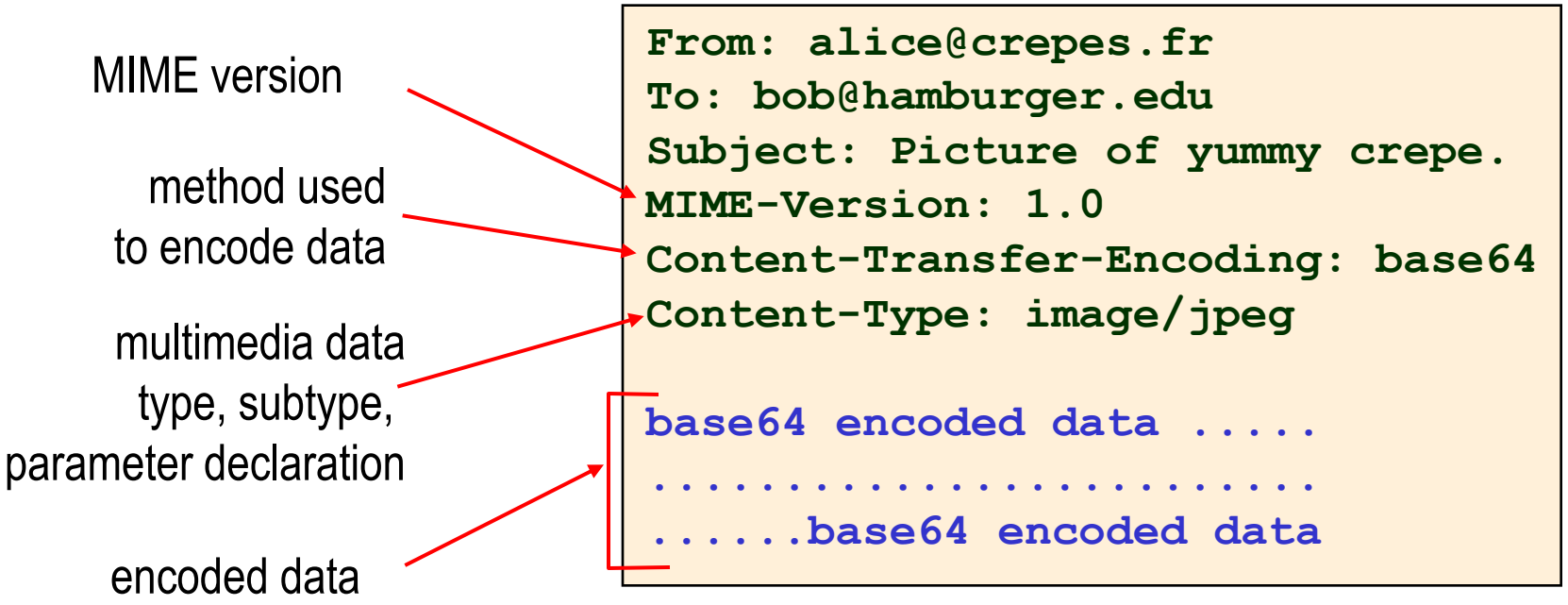
- header lines, e.g.,
  - To:
  - From:
  - Subject:

  different from SMTP commands!
- body
  - the "message", ASCII characters only!!!



Message format: multimedia extensions

- MIME: multimedia mail extension, RFC 2045, 2056
- additional lines in msg header declare MIME content type



MIME types

`Content-Type: type/subtype; parameters`

Text

- example subtypes: **`plain, html`**

Image

- example subtypes: **`jpeg, gif`**

Audio

- exampe subtypes: **`basic`** (8-bit mu-law encoded), **`32kadpcm`** **(32 kbps coding)**

Video

- example subtypes: **`mpeg, quicktime`**

Application

- other data that must be processed by reader before "viewable"
- example subtypes: **`msword, octet-stream`**

Multipart Type

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary=98766789

--98766789
Content-Transfer-Encoding: quoted-printable
Content-Type: text/plain

Dear Bob,
Please find a picture of a crepe.
--98766789
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.........................
......base64 encoded data
--98766789
```

The received message

```
Received: from crepes.fr by hamburger.edu; 12 Oct 98
15:27:39 GMT
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
..................................... .....base64
encoded data
```

Did anyone open my email?

```html
<!DOCTYPE html>
<html>
  <body>
    <p>Hello,</p>
    <p>This is a test HTML email with a tracking
        pixel.</p>

    <!-- Invisible tracking pixel -->
    <img src="https://tracking.example.com/pixel
        ?id=12345"
         alt=""
         width="1"
         height="1"
         style="display:none; border:0;">

  </body>
</html>
```

Hello,

This is a test HTML email with a tracking pixel.

No one can see this –
It's a single pixel at a random location

An image stored on MY server

/pixel

Log access

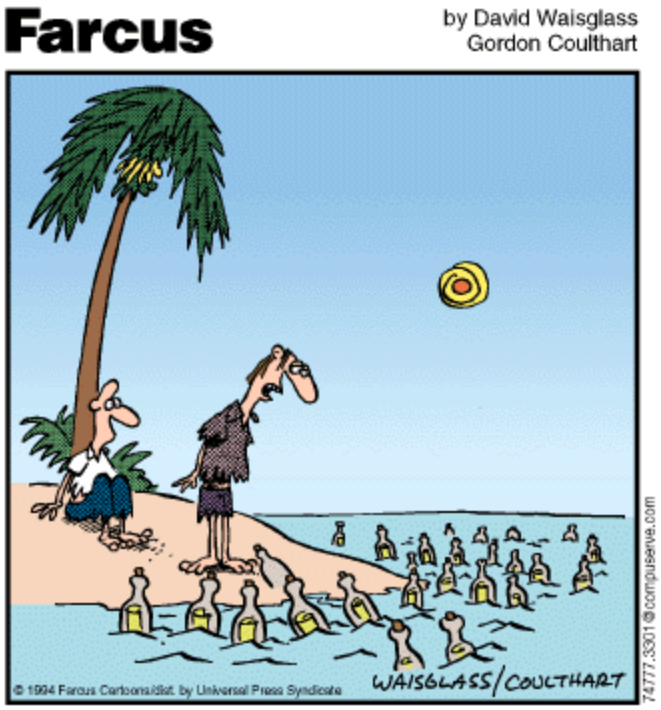
Check status

Mail client:

Mail was opened

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN



"We must be on a mailing list."

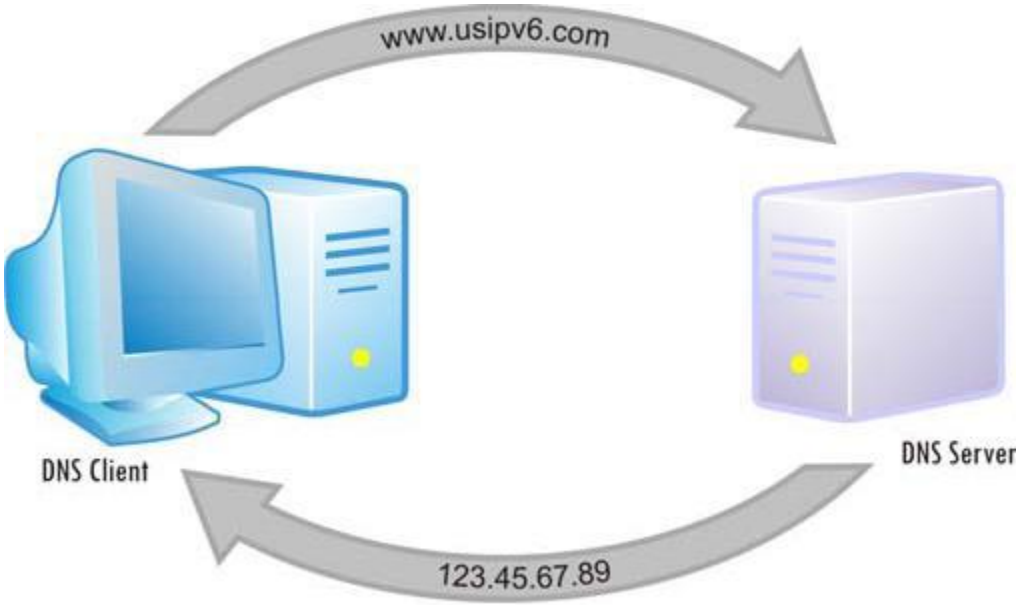
Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

שרות תרגום שמות

DNS: Domain Name System

The internet's directory service – translates hostnames to IP addresses



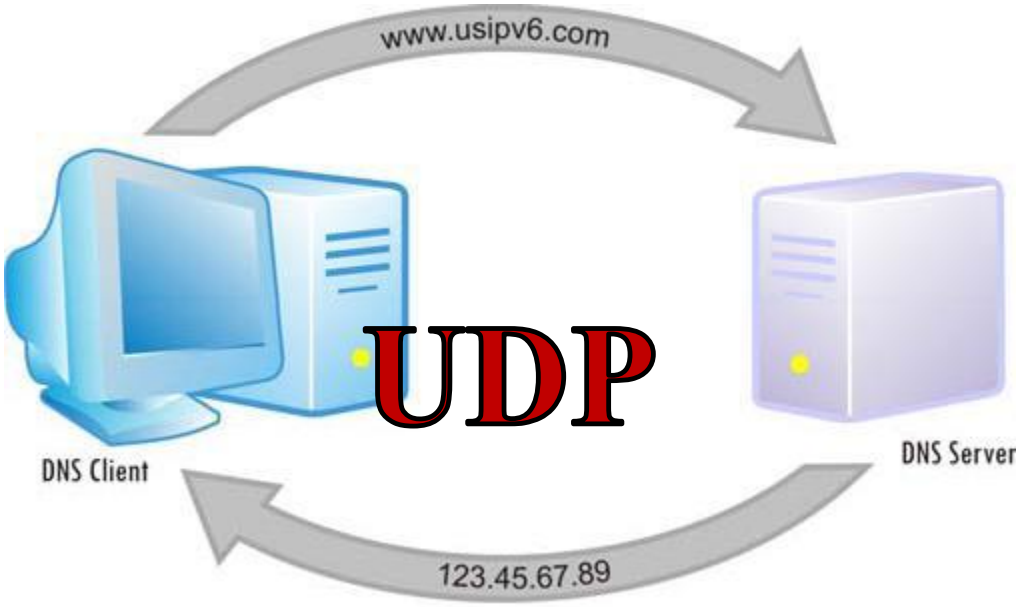
Domain Name System

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol* -- communicate to *resolve* names

שרות תרגום שמות

DNS: Domain Name System

The internet's directory service – translates hostnames to IP addresses



Domain Name System

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol* -- communicate to *resolve* names

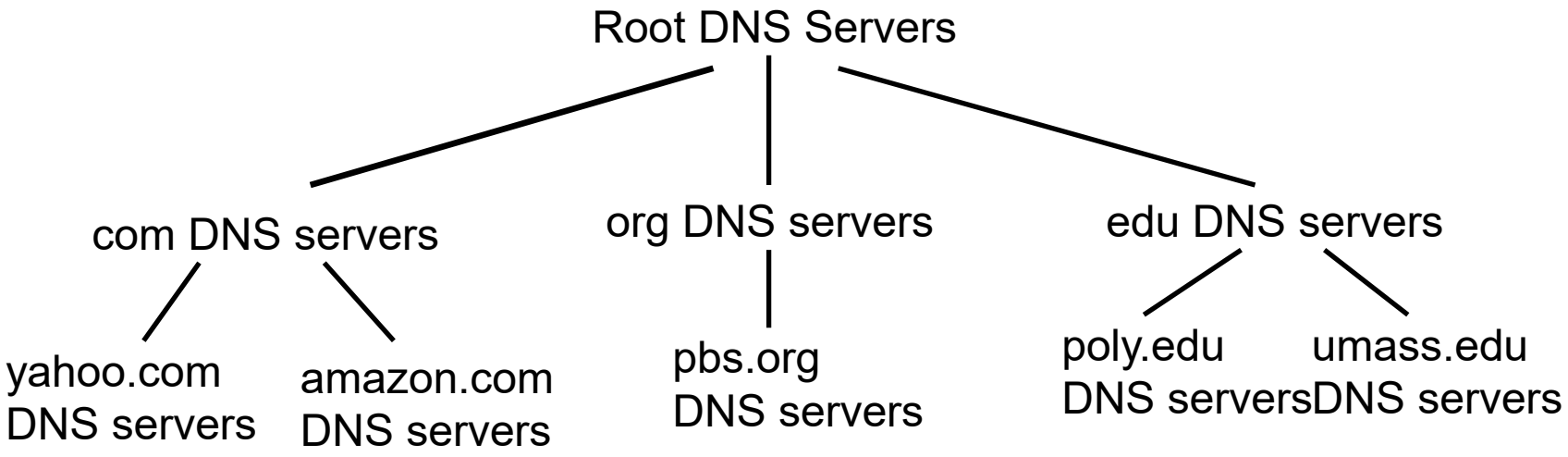


Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

doesn't *scale!*

DNS: a distributed, hierarchical database



Root DNS Servers

Top-level domain (TLD) servers

Authoritative DNS servers

Local DNS server

DNS: root name servers

13 root name "servers" worldwide



TLD, authoritative servers

top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- Network Solutions maintains servers for .com TLD
- Educause for .edu TLD

authoritative DNS servers:

- Organizations own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

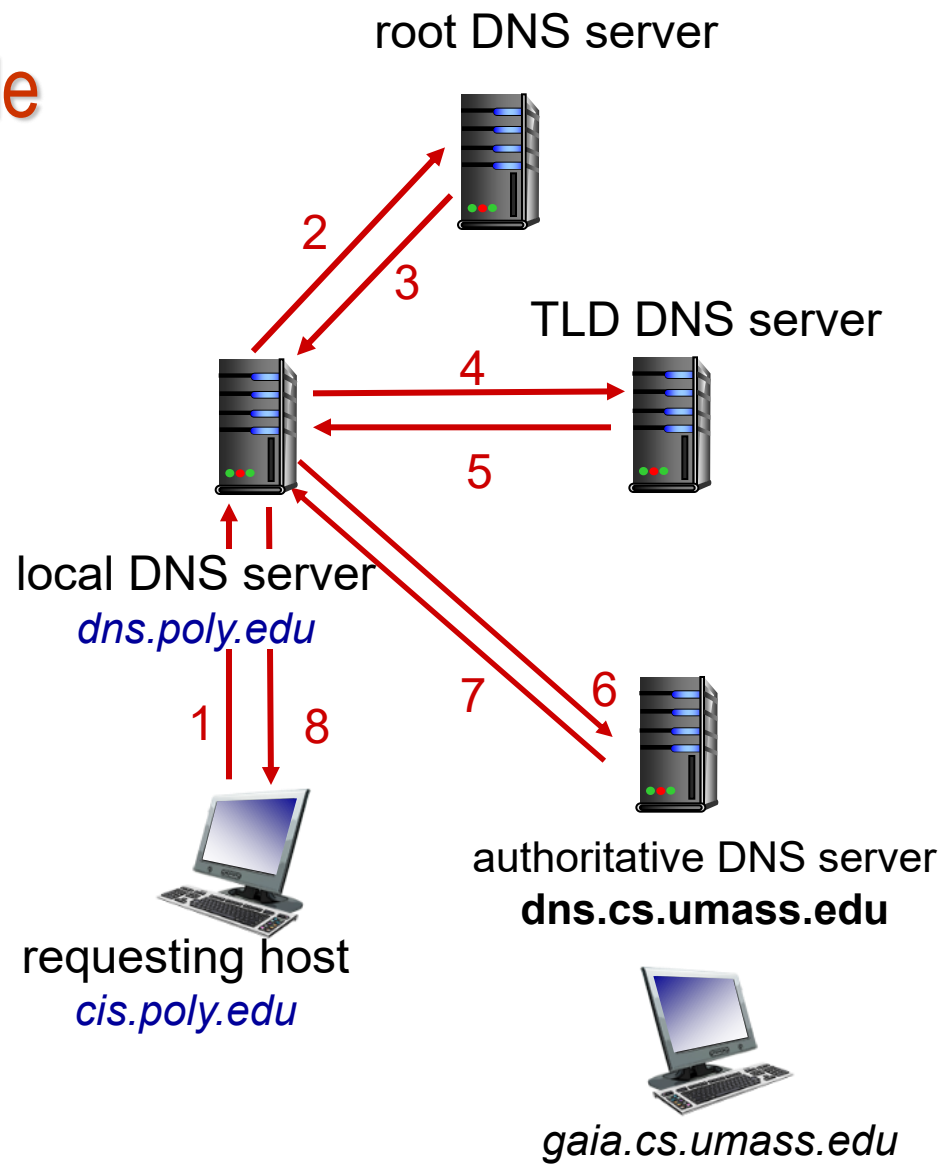
Local DNS name server

- does not strictly belong to hierarchy
- each ISP (residential ISP, company, university) has one
  - also called "default name server"
- when host makes DNS query, query is sent to its local DNS server
  - has local cache of recent name-to-address translation pairs (but may be out of date!)
  - acts as proxy, forwards query into hierarchy

DNS name resolution example

iterated query:

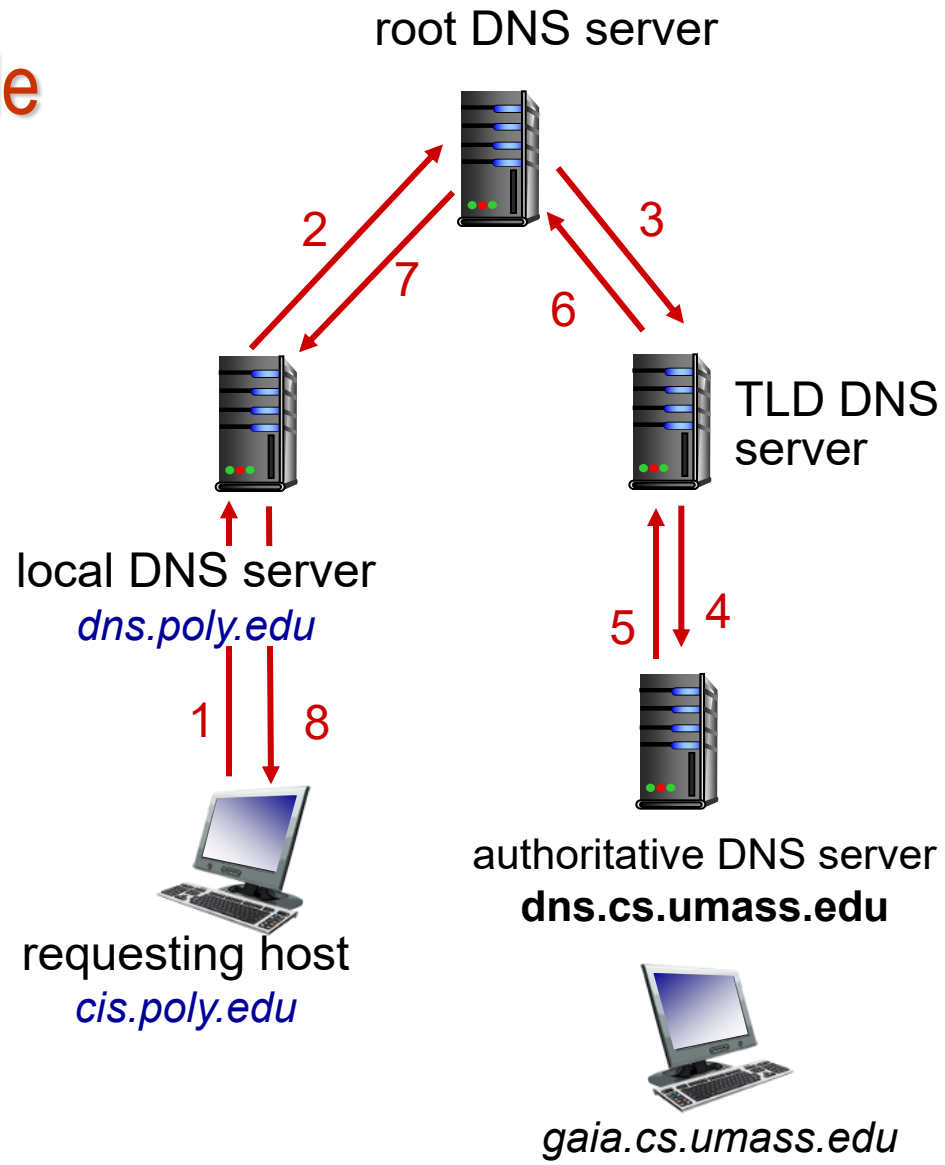
- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



DNS name resolution example

recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



DNS: caching

- Once (any) name server learns mapping, it *caches* mapping
  - cache entries timeout (disappear) after some time
  - TLD servers typically cached in local name servers. Thus, root name servers not often visited



Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - ✓ General description & caching
  - Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

DNS records

DNS: distributed db storing Resource Records (RR)

RR format: **`(name, value, type,TTL)`**

Type=A

- **`name`** is hostname
- **`value`** is IP address
- **(www.foo.com, 145.37.93.126, A)**

Type=NS

- **`name`** is domain (e.g., foo.com)
- **`value`** is hostname of authoritative name server for this domain
- **(foo.com, dns.foo.com, NS)**

Type=CNAME

- **`name`** is an alias name for some "cannonical" (the real) name
- **`value`** is cannonical name
- **(foo.com, relay1.foo.com, CNAME)**

Type=MX

- **`value`** is hostname of mailserver associated with **`name`**
- **(foo.com, mail.bar.foo.com, MX)**

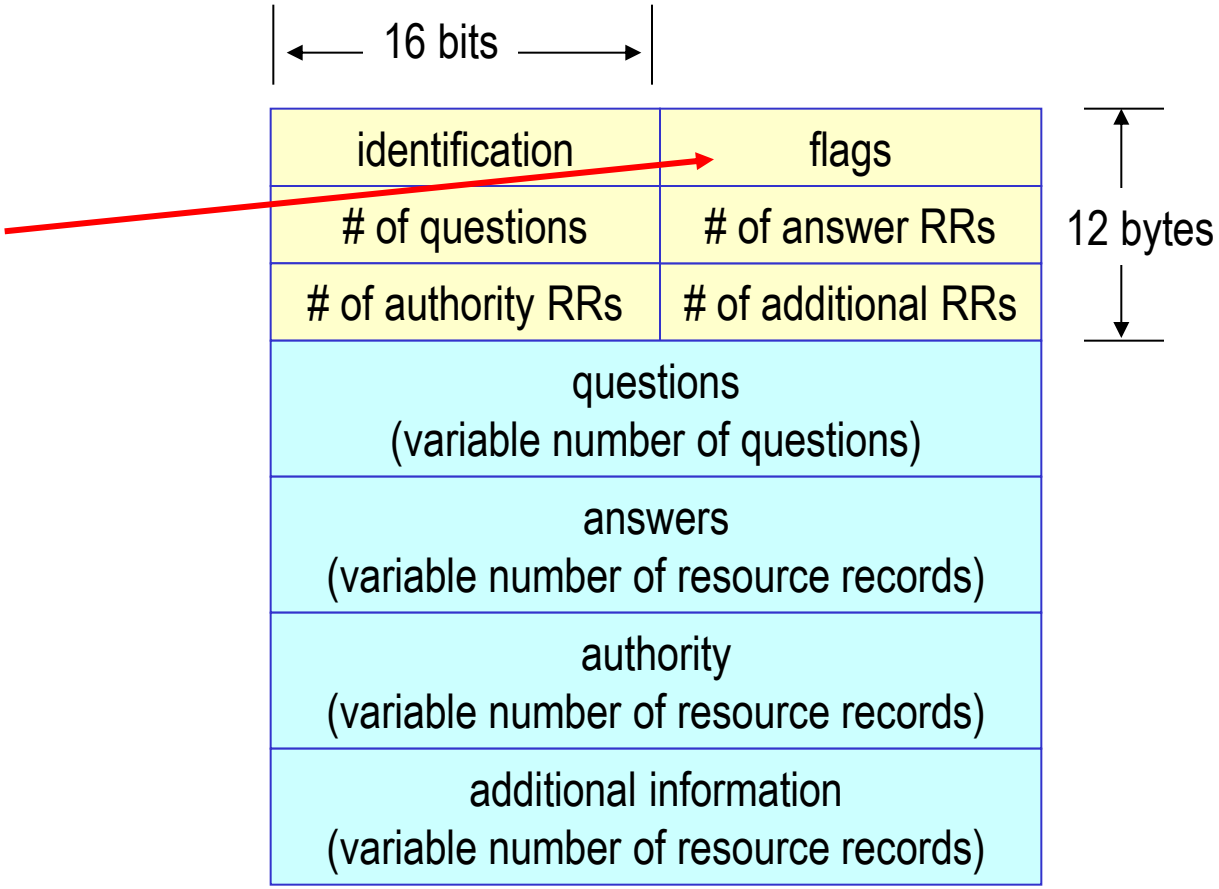
TTL = time to live (determine the time to remove from a cache)

DNS protocol, messages

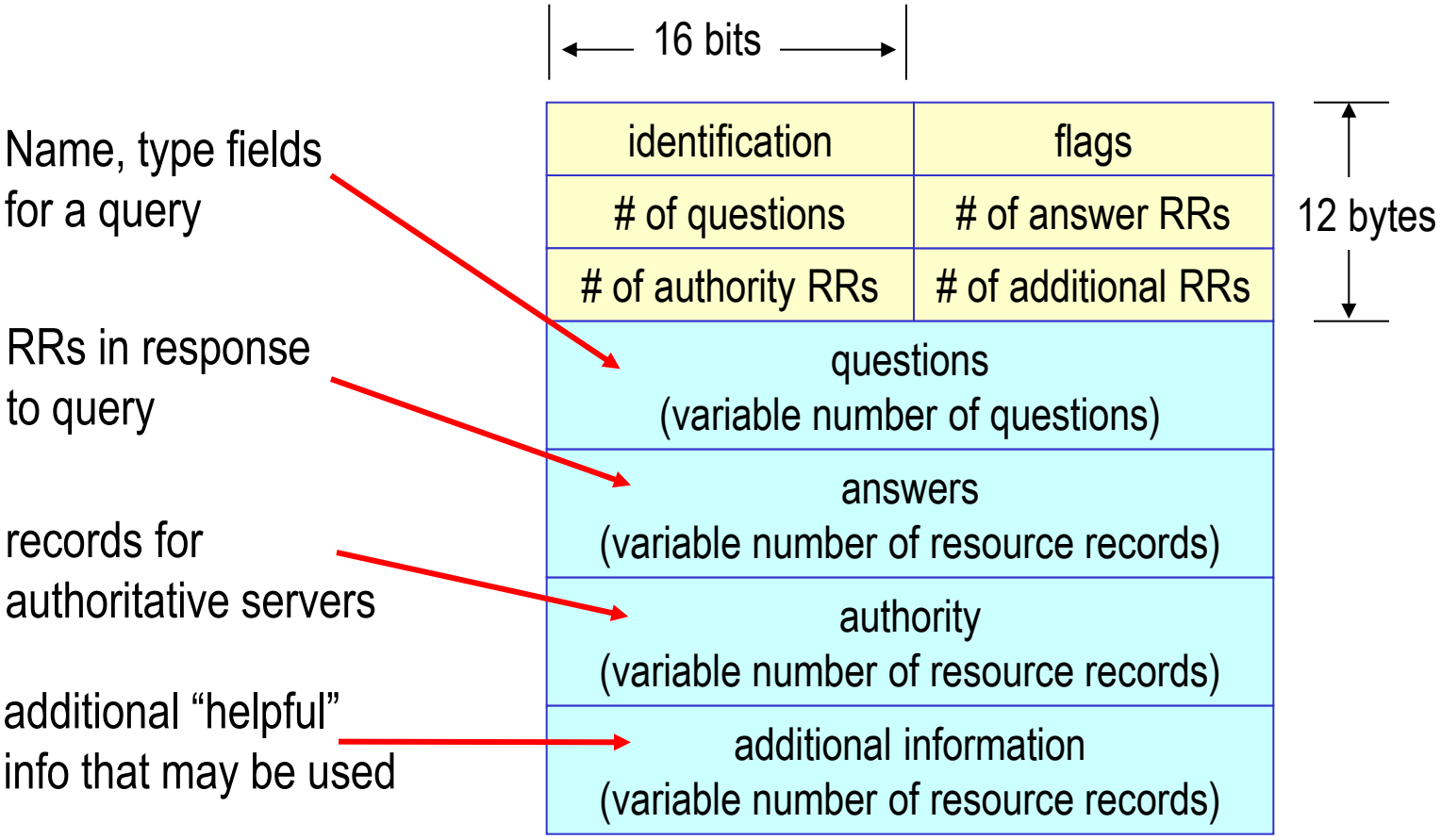
DNS protocol : *query* and *reply* messages, both with same *message format*

Flags:

- query or reply
- recursion desired
- recursion available
- reply is authoritative



DNS protocol, messages



Inserting records into DNS

- Example: just created startup “Network Utopia”
- Register name networkuptopia.com at a registrar (e.g., Network Solutions)
  - Need to provide registrar with names and IP addresses of your authoritative name server (primary and secondary)
  - Registrar inserts two RRs into the com TLD server:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- Put in authoritative server Type A record for www.networkuptopia.com and Type MX record for networkutopia.com

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - DNS lookup
  - DNS privacy
  - CDN

DNS lookup - nslookup

```
Select C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.2251]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>nslookup www.runi.ac.il
Server:  UnKnown
Address:  2a06:c701:436c:5200:2b8:c2ff:fea1:1245

Non-authoritative answer:
Name:    www.runi.ac.il
Address:  194.153.101.3


C:\Users\User>nslookup www.runi.ac.il 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    www.runi.ac.il
Address:  194.153.101.3

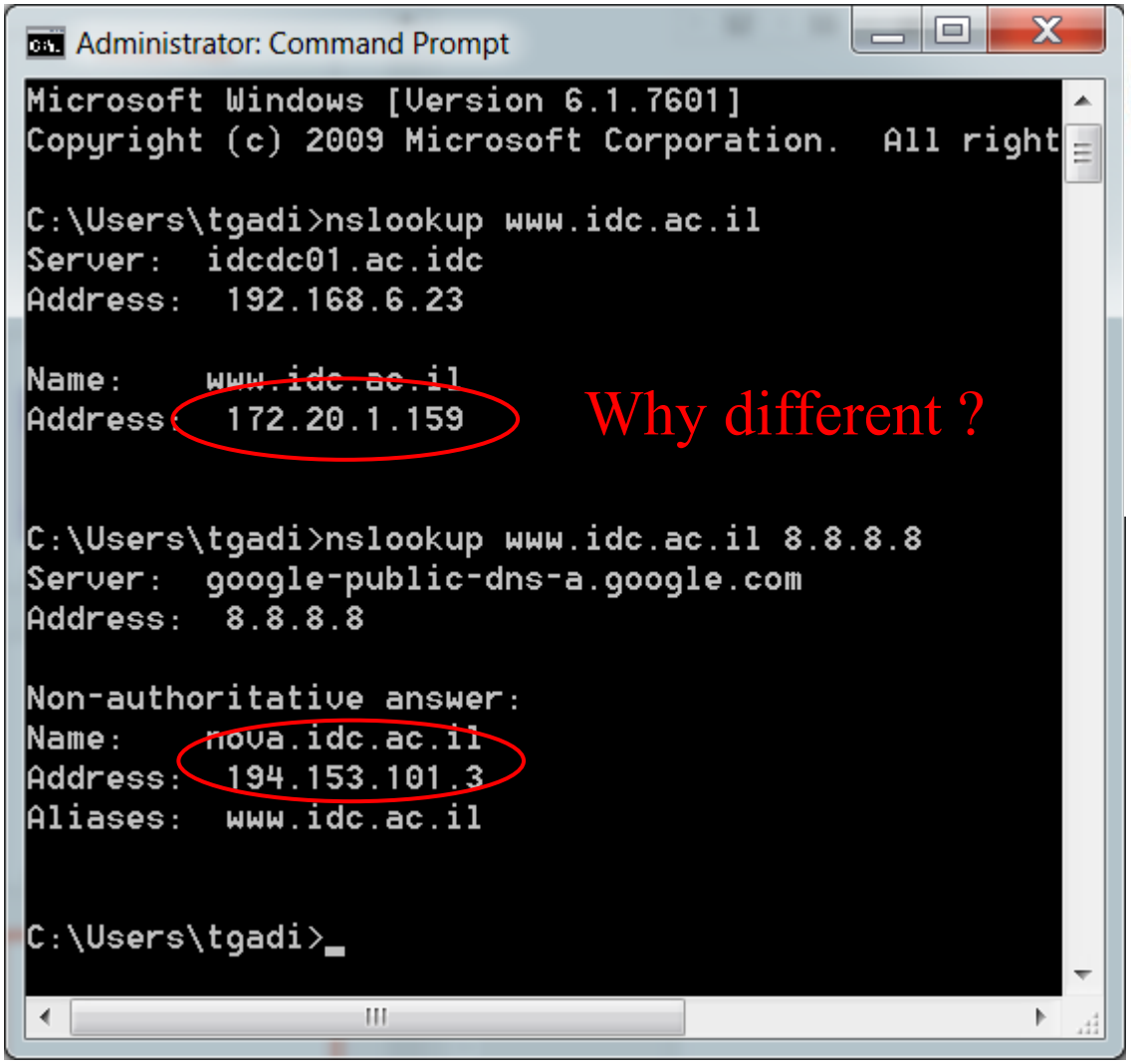

C:\Users\User>
```







DNS lookup split view



Why different ?







Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - DNS privacy
  - CDN

DNS Privacy

Encryption between DNS client and DNS resolver

An IETF study in July 2019 found that:

- July 2019: 8.5% of the Networks intercept DNS queries (27.9% in China)
- (today) most queries are answered un-altered, but the situation might change, intercept server might change DNS answers.
- SOLUTIONs?
  - DNS over TLS (DoT). RFC 7858 (May 2016); RFC 8310 (March 2018)
  - DNS over HTTPS (DoH). RFC 8484 (October 2018)



DoT vs DoH

Category	**DoT (DNS over TLS)**	**DoH (DNS over HTTPS)**
How it works	Encrypts DNS directly using TLS on port **853**	Sends DNS inside **HTTPS (HTTP/2/3)** on port **443**
Traffic appearance	Looks like encrypted DNS	Looks like regular HTTPS web traffic
Blocking / Filtering	Easy to block (dedicated port)	Hard to block; indistinguishable from normal HTTPS
Performance	Lower overhead (simple TLS)	Higher overhead (HTTP + TLS + headers)
Network compatibility	May be blocked on restrictive networks	Works even on very restricted networks (hotspots, corporate)
Control / Monitoring	Easier for enterprises to inspect/log	Harder; can bypass OS and enterprise DNS policies
Privacy benefits	Encrypts DNS, but visible as DNS	Encrypts DNS and hides it inside web traffic

DNS Privacy

DNS over TLS (DoT) and DNS over HTTPS (DoH)



Differences between DoT and DoH

- DoT can be easily blocked, because it is running on a dedicated port (853)
- DoH is made to look like normal HTTPS traffic (port 443), selective blocking of DoH is difficult.
- DoH enables developers to do DNS name resolutions as part of their application (Chrome, Firefox), which some people think is very bad.
- Adopted by Firefox and Chrome (but not as default).

Interesting talk about this topic (September 2019):
https://www.youtube.com/watch?v=pjin3nv8jAo
(~ 40 minutes)

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - CDN

Content Distribution Networks (CDNs)



Content Distribution Networks (CDNs)

- Video traffic: major consumer of Internet bandwidth

- Challenge: scale - how to reach ~1B users?







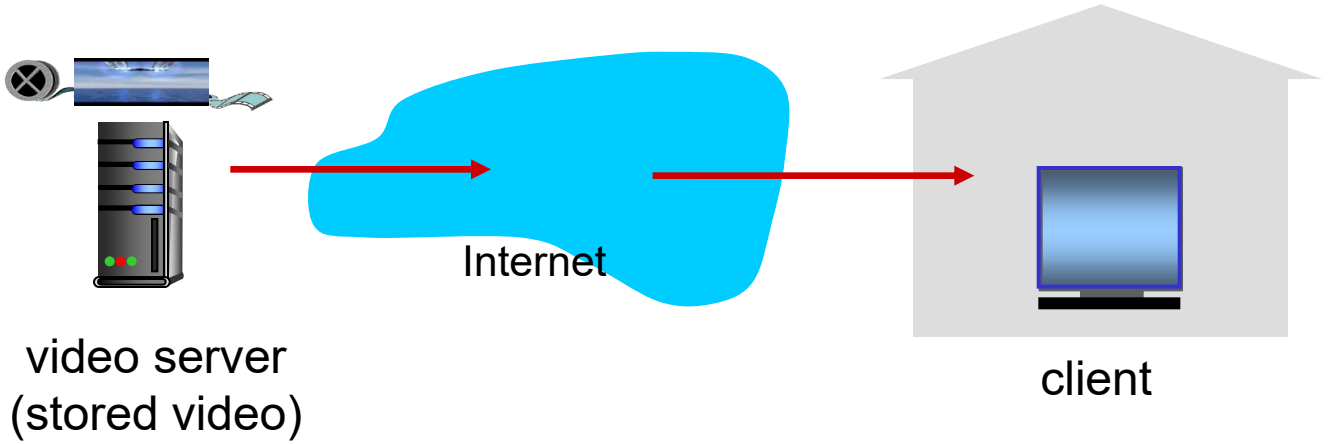






Streaming stored video

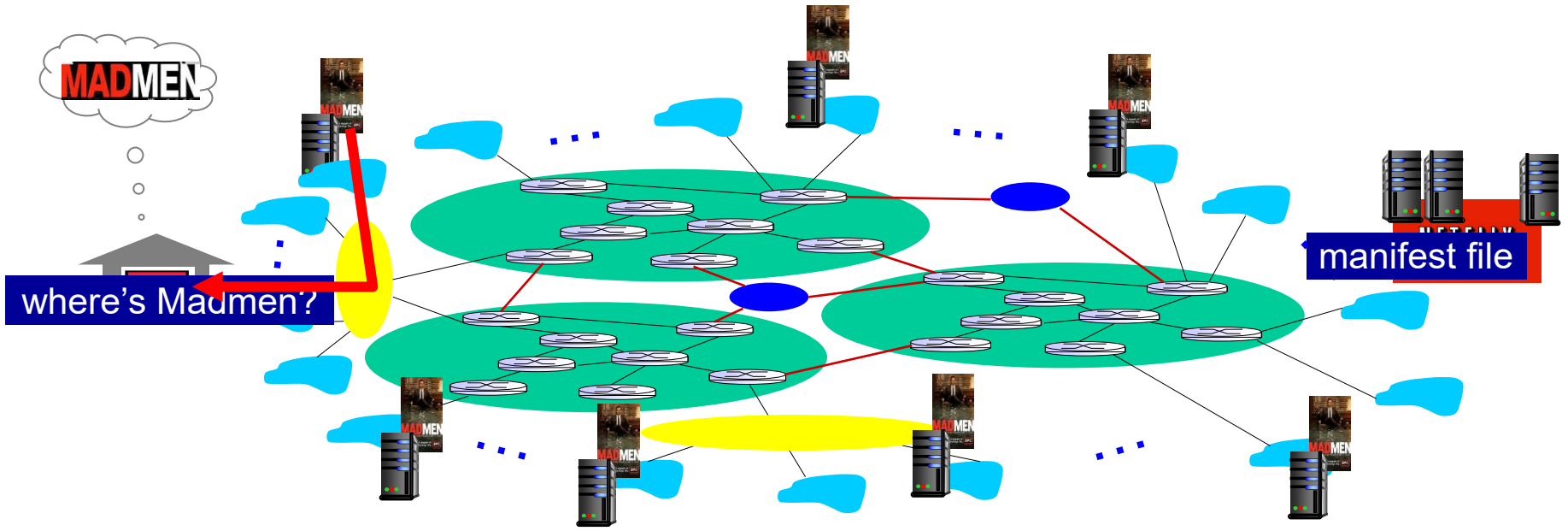
simple scenario:



Single mega-video server won't work (why?)

Content Distribution Networks (CDNs)

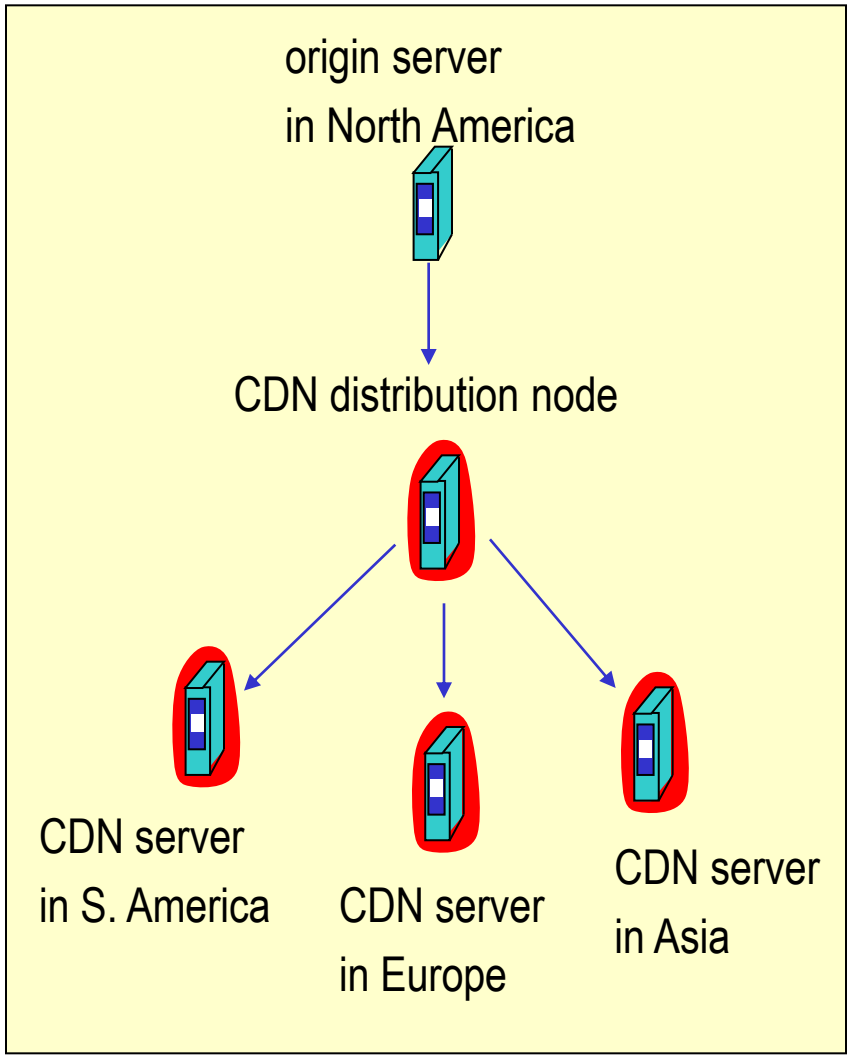
CDN: stores copies of content at CDN nodes
e.g. Netflix stores copies of MadMen



Content Distribution Networks (CDNs)

Content replication

- CDN company installs hundreds of CDN servers throughout Internet (in lower-tier ISPs, close to users)
- CDN replicates its customers' content in CDN servers. When provider updates content, CDN updates servers
  - replicate all content
  - replicate partial content
- The content providers are the CDN customers
- Make use of DNS name resolution!



Content Distribution Network

הסיפור של חברת אקמאי





Daniel M. Lewin
1970-2001
Akamai Co-Founder and
Chief Technology Officer

"Akamai's mission is to transform the delivery of Internet content so that Web congestion is a thing of the past."

Today

- ✓ Web browsing protocol: HTTP
  - ✓ Proxy
  - ✓ Security
  - ✓ HTTP 2 & HTTP 3
- ✓ Mail Protocols:
  - ✓ SMTP
  - ✓ POP3
  - ✓ IMAP
- ✓ DNS:
  - ✓ General description & caching
  - ✓ Actual messages
  - ✓ DNS lookup
  - ✓ DNS privacy
  - ✓ CDN